Code-Guided Video Editing with Source-Grounded Executable Proxies

Yuanhong Zheng
Peking University



Figure 1. **An executable plan for editing an existing video.** Given the source and an instruction to add a wall clock, a coding agent builds a procedural scene that makes the intended placement explicit. A frozen video model combines this proxy with the source video to produce the edit. The no-proxy result adds an oversized clock; proxy guidance follows the planned placement and scale more closely. All four views are taken at the same time in the wall clock addition example.

Abstract

Video editing requires translating a user's intent into coordinated changes to objects, scene layout, camera framing, and temporal behavior while retaining the relevant content of the source video. We explore executable scene programs as an intermediate interface for this task. Given a source video and an editing instruction, a video-aware coding agent constructs a procedural scene that expresses the intended edit and renders it into a proxy video. The resulting program exposes a concrete editing plan that can be executed, inspected, and refined before final synthesis. We instantiate this approach as a training-free system built on a frozen reference-conditioned video generator. The source video remains available throughout sampling, while the edited proxy participates only in the early sampling steps. Selected examples from all eight OpenVE-Bench editing categories illustrate object addition and removal, local changes, background replacement, camera reframing, stylization, creative transformations, and subtitle removal. Matched no-proxy comparisons highlight improvements in edit localization, source-content retention, and target framing in these examples. These examples establish a practical interface between programmatic editing and generative video synthesis. We further describe how executable proxies could support the construction of structured editing data for subsequent model training.

1. Introduction

Editing a video often begins with a short instruction: add a bird in the upper-left sky, remove a person from a room, or widen the view of a clock tower. Each request implies a concrete intervention in a scene. An added object needs a location and behavior over time; a removal needs a plausible account of the newly visible background; a framing change needs a camera plan. At the same time, the source footage supplies content that the creator wants to retain, including the identity of the scene, the subject's performance, and the timing of the shot. An effective editing interface should make the intended intervention explicit while continuing to use this source information.

Diffusion-based visual synthesis has developed powerful generative priors [10, 22]. Video editing methods build on these priors through attention reuse, feature propagation, reference conditioning, and learned editing models [7, 12, 14, 21]. These advances make a broad range of edits possible. They also motivate a complementary question: what representation should a creator or an agent use to specify the target change before the final video is synthesized? For edits involving layout, object relations, or camera framing, an executable representation offers a direct way to express and inspect the plan.

A scene program can instantiate objects, assign their attributes, define a camera, and evaluate their states at a chosen time. Rendering that program produces a visual preview of the requested edit. The preview makes the relation between an instruction and its spatial and temporal consequences visible. For example, the added clock in Fig. 1 has an explicit position relative to the person and the wall before the final video is generated. The program also remains available for inspection and revision, providing an artifact alongside the rendered result.

We study this interface for editing existing videos. The source video and the instruction are given jointly to a coding agent, which produces a procedural scene for the edited sequence. The scene focuses on the composition, entities, timing, and target changes needed by the task. Its render serves as an edited proxy video. A reference-conditioned video generator then receives the source, the edit instruction, and the proxy. This division gives the source and the proxy distinct roles: the source anchors the original scene and performance, and the proxy expresses the intended intervention. We preserve the source condition throughout sampling and provide the proxy only during the initial steps.

Our implementation builds on Code Video Model's executable video previews and early-step conditioning [15]. We adapt this foundation to source-video editing by grounding program construction in observed footage and retaining that footage as a persistent reference during synthesis. The current system requires no additional training of the video generator. We present successful examples drawn from the eight editing categories of OpenVE-Bench [8]. The examples expose the full source–program–proxy–output chain and illustrate how one executable interface can express varied editing requests. This technical report focuses on the method and its qualitative applications; it also outlines a path from the current system to structured editing data and future learned models.

Our contributions are:

- A source-video editing interface in which a coding agent translates observed footage and an instruction into an executable scene program.
- A training-free realization that combines a persistent source-video reference with an edited proxy under an early-step conditioning schedule.
- Paired qualitative demonstrations spanning all eight OpenVE-Bench categories, with both the executable proxy and a matched no-proxy result visible for every main-gallery example.

2. Related Work

Video editing with generative priors. Tune-A-Video adapts an image diffusion model to a video using one-shot tuning [26]. FateZero reuses attention information obtained from the source video [21], and TokenFlow propagates diffusion features using inter-frame correspondences [7]. FLATTEN introduces optical-flow-guided attention [4], while RAVE combines frame interactions with randomized noise shuffling [13]. CoDeF represents a video through canonical content and temporal deformation, allowing image edits to be propagated across frames [20]. AnyV2V uses an edited first frame and feature injection into an existing image-to-video model [14]. These methods provide several ways to carry source information through editing. Our system adds an executable target-scene preview as a separate conditioning input to a pretrained video generator.

Explicit control for visual synthesis. Structure- and content-guided video synthesis uses distinct conditions to control these aspects of an edit [5]. ControlNet demonstrates the utility of spatial conditions such as edges, depth, and pose for image generation [29]. VideoComposer combines textual, spatial, and temporal conditions for video synthesis [24]. MotionCtrl separates camera and object motion control [25], and CameraCtrl parameterizes camera trajectories for controllable video generation [9]. VACE organizes references, video inputs, and masks within a unified creation and editing framework [12]. ReVideo combines first-frame content editing with trajectory-based motion control [18]. Our programmatic proxy can jointly render layout, camera, object behavior, and semantic changes into a video condition. The underlying generator in our implementation is the reference-conditioned MiniMax-H3 model [17].

Programs as visual representations. Holodeck generates three-dimensional environments from language and spatial constraints [28]. BlenderAlchemy uses vision-language feedback to search for graphics-editing actions [11]. The Scene Language represents scenes through programs, semantic descriptions, and identity embeddings [30]. Code-as-World investigates executable representations for physical reasoning [23]. For video creation, Code2Video generates educational animations through executable code [2]. Code2Video-Compass evaluates programmatic video generation across text, image, and video inputs [3]; recent open-model projects also explore code-driven video creation [19]. These works establish a broad context for executable visual interfaces. Our focus is the use of a scene program to communicate an edit to an existing source video.

Code Video Model is the closest foundation for our implementation [15]. It renders executable scene programs into video previews and introduces early-step conditioning to reduce transfer of a proxy's rendered appearance. It also demonstrates scene-editing applications. We reuse these ideas and study a source-grounded setting in which ob-

served video, rather than an existing procedural scene, is the starting point. The original video remains a reference during synthesis.

Editing data and evaluation. InstructPix2Pix shows how generated editing pairs can support instruction-based image editing [1]. InsViE-1M constructs video-editing triplets with dedicated filtering stages and trains an instruction-following editor [27]. OpenVE-3M provides a large video-editing dataset and OpenVE-Bench, whose eight categories supply the source clips and requests used in our showcases [8]. OmniEdit-Bench and VEFX-Bench further emphasize video-specific editing coverage, instruction fidelity, and preservation assessment [6, 16]. Our present study uses selected qualitative examples. Program-conditioned data construction and broader evaluation are natural next steps for this interface.

3. Method

3.1. Editing through an executable intermediate

Let $V_{\text{src}} = \{I_i\}_{i=0}^{N-1}$ denote a source video and e an editing instruction. The desired output V_{edit} should realize the requested change while retaining the source content relevant to the task. We express the target edit through a scene program P:

$$P = \mathcal{A}(V_{\text{src}}, e), \tag{1}$$

$$V_{\text{proxy}} = \mathcal{R}(P), \tag{2}$$

$$V_{\text{edit}} = \mathcal{G}(V_{\text{src}}, e, V_{\text{proxy}}). \tag{3}$$

Here $\mathcal{A}$ is a video-aware coding agent, $\mathcal{R}$ executes and renders the program, and $\mathcal{G}$ is a frozen reference-conditioned video model. Figure 2 summarizes the implementation.

The program describes the scene needed to visualize the requested intervention. For a local addition, it specifies the new object's placement and behavior alongside an approximation of the surrounding scene. For a camera edit, it specifies the target framing and the relevant environment. For a removal, it renders the scene without the target entity. This task-dependent construction allows the same interface to express different editing operations. The source video continues to supply appearance and motion information that the procedural approximation may represent only coarsely.

3.2. Source-grounded scene construction

The coding agent receives the edit request, the source video, and time-indexed source frames. It inspects these observations and constructs a Three.js scene for the edited sequence. The task contract asks the agent to retain the original composition, camera, occlusion relationships, action timing, and shot structure wherever the edit does not request a change. The requested change is implemented directly in the generated scene. The agent starts from the footage; no pre-existing procedural counterpart of the source is required.

The program can encode several complementary aspects of an edit. Object groups and geometry expose the entities and their relative layout. Camera parameters expose the viewing configuration. Time-dependent transforms expose motion and event timing. Materials, lighting, and procedural effects communicate semantic attributes or requested appearance changes. These components are ordinary program constructs rather than a fixed scene-graph schema. They can be inspected and modified in the generated code.

For the clock in Fig. 1, the program specifies both its placement on the wall and its animation over time.

We use a fully procedural rendering contract. Source frames are available for observation, but are not inserted into the render as image textures, video textures, background plates, or composited overlays. The proxy therefore provides an explicit visual account of the target scene. Photographic detail is supplied by the video generator using the original source as a reference. Figure 1 illustrates this separation: a procedural person, room, and clock establish the edit plan, while the synthesized output retains the photographic character of the original scene.

3.3. Execution, inspection, and temporal alignment

Each generated scene exposes a deterministic evaluation interface `seek(t)`. Calling this interface updates and renders the scene at time t, including camera state, object transforms, visibility, and any procedural animation. Evaluating timestamps out of order must produce the same corresponding states. This property makes the proxy reproducible and supports visual inspection at arbitrary times.

The agent renders keyframes and contact sheets, compares them with time-matched source observations and the edit request, and refines the program. The inspection contract covers camera and composition; geometry and occlusion; motion and timing; and material contrast and semantic legibility. The final encoded proxy is inspected again. Execution checks also verify the target resolution, frame count, silent proxy output, and per-frame camera metadata. These checks establish that the program can serve as a well-formed condition; visual inspection assesses whether it communicates the intended edit.

We render the source-conditioned proxy on the same timeline as the normalized source video. Aspect ratio and composition are retained through proportional resizing and padding to 960×544 at 24 frames per second. When the generator requires additional alignment frames, we append copies of the final state to both conditions. Published results are trimmed back to the visible source interval. This avoids introducing new action into the alignment tail.

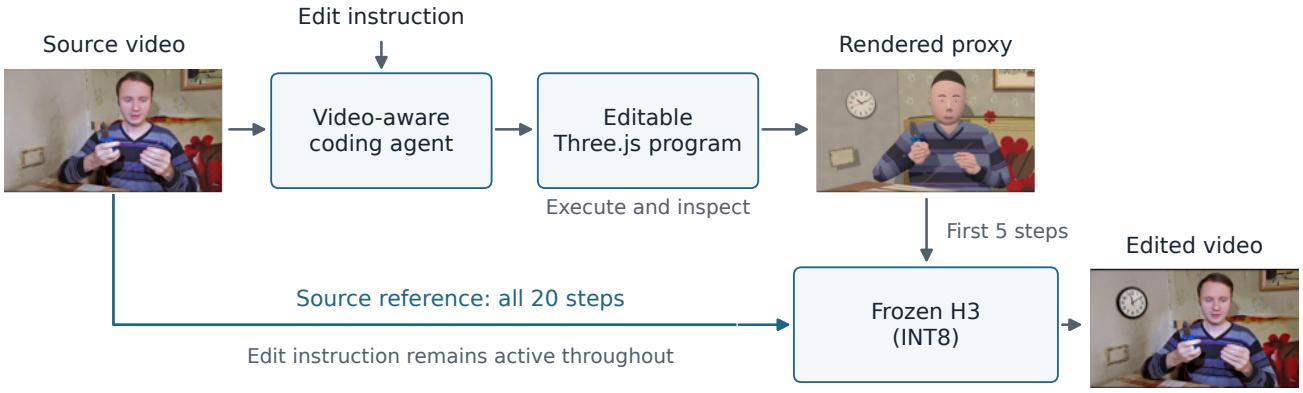


Figure 2. **Source-grounded executable video editing.** A coding agent observes the source and the edit request, constructs an editable scene program, and inspects its render. The resulting proxy and the source have separate roles during synthesis: the proxy is active for the first five steps, while the source reference and edit instruction remain active throughout the 20-step trajectory. All model weights remain frozen.

3.4. Early-step proxy conditioning

We use MiniMax-H3's reference-conditioned generation interface [17]. The text identifies `<Video 1>` as the source and `<Video 2>` as the edited structural proxy. It asks the model to use the source for the original scene and to use the proxy for the requested intervention, subject to the edit instruction. The source reference remains active throughout sampling.

Following Code Video Model's early-step conditioning principle [15], we limit proxy conditioning to the initial sampling steps. With zero-based step index k, total step count K, and proxy budget K_e, the conditioning state is

$$c_k = \begin{cases} \mathrm{Enc}(e, V_{\mathrm{src}}, V_{\mathrm{proxy}}), & 0 \le k < K_e, \\ \mathrm{Enc}(e, V_{\mathrm{src}}), & K_e \le k < K. \end{cases} \tag{4}$$

The encoder notation includes both multimodal semantic conditioning and variational autoencoder (VAE) reference conditioning. Our implementation uses $K = 20$ and $K_e = 5$. At the transition, we switch from the source-plus-proxy conditioning state to the source-only state in both pathways. The source VAE representation is shared between these states. The same edit instruction is retained.

Sampling follows one continuous latent trajectory. The switch changes the active references and their associated layout; it does not restart sampling or re-noise the latent. Thus, the proxy can influence the developing edit early in the trajectory, and later steps continue with the source as the visual reference. All weights of the video generator and its encoders remain frozen. The coding agent's program construction and visual refinement also require no task-specific model training.

4. Qualitative Results

4.1. Sources and implementation

We use source clips and editing requests from OpenVE-Bench [8]. The gallery presents one selected successful example from each of its eight categories, together with an introductory wall-clock example. All images come from the existing training-free early-step runs. The coding stage uses a multimodal coding agent, and synthesis uses a quantized MiniMax-H3 implementation with 20 Euler steps, seed 42, and proxy conditioning for the first five steps. The source remains available for all 20 steps. No additional model training or video generation was performed to construct the figures. The gallery is intended as a capability demonstration using selected outputs, rather than an aggregate benchmark comparison.

For every example, we show four time-aligned views: the source, the executable proxy, the output without a proxy, and the output with early-step proxy guidance. Both output groups use the same normalized source, main edit request, model, seed, and sampling settings. The no-proxy group uses the source throughout sampling; the guided group adds the proxy during the first five steps. The figure instructions are concise summaries of the original requests. Complete input prompts and generation settings are retained in the underlying run records; Appendix A gives reproduction details. All displayed frames preserve the full normalized composition.

4.2. Entity and spatial edits

Local addition. The seagull addition example places an animated bird in the upper-left sky. The procedural scene contains the coastline and boat as context, with the new bird placed above them. The output in Fig. 3 introduces

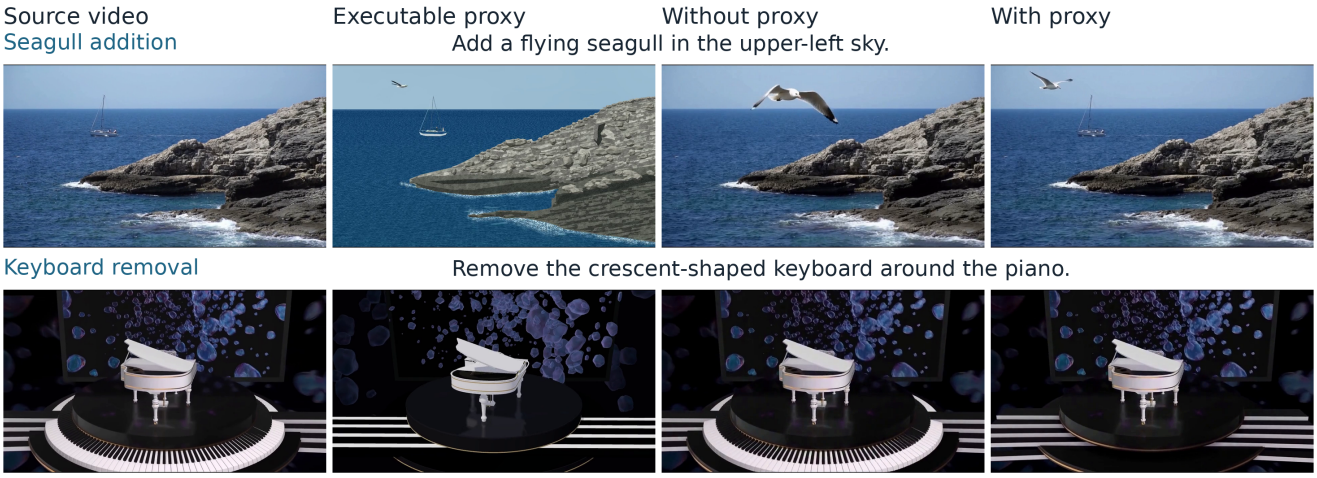


Figure 3. **Adding and removing entities.** Top: a seagull is added over the coastal scene. Proxy guidance keeps the bird closer to the planned scale and location, whereas the baseline places a much larger bird in the foreground. Bottom: the crescent-shaped keyboard around the piano is retained by the baseline and removed with proxy guidance.

the bird while retaining the recognizable rocky shore and sea. The clock example in Fig. 1 offers a complementary indoor addition: the proxy locates a new object relative to an existing person and wall. These examples show how an addition can be made explicit before synthesis. The no-proxy outputs introduce the requested objects but overshoot the planned scale or placement; the guided outputs follow the corresponding proxy plans more closely.

Local removal. The keyboard removal example targets the crescent-shaped keyboard surrounding a grand piano. The baseline retains this prominent keyboard ring. The proxy explicitly omits the ring while retaining the central piano, stage, and surrounding visual arrangement. The guided output follows this removal (Fig. 3), illustrating how the rendered target scene can make an omission concrete.

Local change. The person aging example replaces a woman with a silver-haired older woman while retaining her pose and position. Both output groups perform the age edit. The baseline additionally changes her white jacket into a dark dress and introduces a large necklace. The proxy-guided result retains the original outfit and phone action (Fig. 4). This comparison illustrates a more targeted intervention with fewer unrequested appearance changes.

Background change. The newsroom replacement example changes the background to a futuristic cyber command center. The baseline realizes the new environment but removes the foreground woman. The proxy includes both the speaker and the target surroundings, and the guided output retains her while changing the background. This example highlights the distinct roles of a requested background change and a foreground subject that should remain in the shot.

4.3. Camera and creative edits

Camera edit. The clock tower example requests a wide shot. The agent constructs the tower and surrounding context under the target framing. The no-proxy output remains close to the original medium view. In the guided output, the tower occupies less of the frame and more of the adjacent environment becomes visible (Fig. 5). The program makes the framing choice explicit through its camera and scene configuration. The requested wide composition applies to the edited shot; it is not an instruction to add a new reveal or a new cut.

Creative edit. The waterfront example transforms a Ferris wheel into a celestial portal. The proxy contains a luminous circular structure and its reflection, providing a spatially organized account of the transformation. The no-proxy result leaves elements of the wheel structure within the transformation. The guided result forms a coherent blue-violet portal in the waterfront setting with reflected light on the water. The example shows how procedural geometry and effects can communicate a creative target while the source supplies scene context.

4.4. Global style and subtitle edits

Global style. The portrait example requests a Pop Art treatment of a close-up profile. The proxy renders the face with graphic contours, flat colors, and halftone details. The baseline mainly applies saturated colors to photographic texture. The guided result adopts drawn contours and halftone



Figure 4. **Targeted edits with better source-content retention.** Top: both outputs age the woman, but the no-proxy result changes her clothing; proxy guidance retains the original white jacket. Bottom: the baseline generates an empty cyber command center, while the guided result retains the foreground speaker in the new environment.



Figure 5. **Reframing and creative transformation.** Top: the no-proxy output remains close to the source framing; the guided result realizes the requested wider shot of the clock tower. Bottom: the baseline blends a portal effect with the original wheel structure; the guided output forms a distinct celestial portal with a corresponding reflection.

details while keeping the profile composition recognizable (Fig. 6). Here, appearance changes are part of the instruction. The source-retention guidance is explicitly subordinate to those requested changes.

Subtitle edit. The aerial landscape example removes subtitles from the top of the frame. The no-proxy output retains the text band. The proxy depicts the scene without text, and the guided output removes the band while retaining the recognizable fields, mountains, and cloudy sky. The example uses the video generator for the final completion; no separate text-erasure or compositing operation is applied to the displayed output.

Temporal inspection. The examples are generated as videos rather than independent edited frames. Appendix Fig. 7 provides additional timestamps for the seagull and portal examples. These sequences expose the continuing edit and its relation to the original shot.

5. Discussion and Future Work

A controllable intermediate. The executable program exposes the scene plan used to condition the video model. Object placement, camera configuration, and temporal behavior can be checked in its render and revised in code. This visibility is useful even when the final appearance is produced by a generative model. The source video and the



Figure 6. **Stylization and subtitle removal.** Top: the portrait is rendered in a Pop Art aesthetic. The baseline mainly changes photographic colors, whereas the guided output adds graphic outlines and halftone detail. Bottom: the baseline retains the subtitle band, while proxy guidance removes it from the aerial landscape.

rendered plan provide complementary information within one interface.

From a training-free system to editing data. The current implementation also suggests a route to structured training examples of the form $(V_{\text{src}}, e, P, V_{\text{proxy}}, V_{\text{edit}})$. Programs can supply object, camera, and temporal information alongside the rendered condition. Combining these signals with independently screened edited videos could support training a model to distinguish the source's content from the proxy's requested intervention. Data curation and filtering would be central to this process, as demonstrated in instruction-based image and video editing [1, 8, 27]. Generated targets would be quality-filtered pseudo-labels. Any resulting improvement would need to be established through training and held-out evaluation.

Scope and next steps. The current program is an approximate, task-conditioned account of a scene. It does not recover uniquely determined geometry or all of the source's appearance and performance. The generator can also alter details outside the intended edit. Our selected showcases establish feasibility for the displayed requests; they do not measure success frequency across the benchmark. Further work includes broader quantitative paired evaluation, human preference studies, and measurements of the cost of program construction. Editing-specific benchmarks provide useful protocols for separating instruction accuracy, preservation, realism, and consistency [6, 16]. Learning to use the source and proxy jointly is a promising extension of the present interface.

6. Conclusion

We presented a training-free approach to video editing through source-grounded executable proxies. A coding agent converts a source video and editing instruction into a procedural scene whose render makes the target edit visible. A frozen video model combines that proxy with a persistent source reference, using proxy conditioning only during the early sampling steps. Selected examples across the eight OpenVE-Bench categories illustrate the range of edits expressible through this interface.

References

[1] Tim Brooks, Aleksander Holynski, and Alexei A. Efros. InstructPix2Pix: Learning to Follow Image Editing Instructions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18392–18402, 2023.

[2] Yanzhe Chen, Kevin Qinghong Lin, and Mike Zheng Shou. Code2Video: A Code-centric Paradigm for Educational Video Generation. *arXiv preprint arXiv:2510.01174*, 2025.

[3] Zhenghan Chen, Zekai Shao, Xin Lin, Leting Cheng, and Siming Chen. Code2Video-Compass: A Comprehensive Benchmark for General-Purpose Code-Based Video Generation across T2V, I2V, and V2V. *arXiv preprint arXiv:2610.28416*, 2026.

[4] Yuren Cong, Mengmeng Xu, Christian Simon, Shoufa Chen, Jiawei Ren, Yanping Xie, Juan-Manuel Perez-Rua, Bodo Rosenhahn, Tao Xiang, and Sen He. FLATTEN: optical FLow-guided ATTENtion for consistent text-to-video editing. In *International Conference on Learning Representations (ICLR)*, 2024.

[5] Patrick Esser, Johnathan Chiu, Parmida Atighehchian, Jonathan Granskog, and Anastasis Germanidis. Structure and Content-Guided Video Synthesis with Diffusion Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7346–7356, 2023.

[6] Xiangbo Gao, Sicong Jiang, Bangya Liu, Xinghao Chen, Minglai Yang, Siyuan Yang, Mingyang Wu, Jiongze Yu, Qi Zheng, Haozhi Wang, Jiayi Zhang, Jie Yang, Zihan Wang, Qing Yin, and Zhengzhong Tu. VEFX-Bench: A Holistic Benchmark for Generic Video Editing and Visual Effects. *arXiv preprint arXiv:2604.16272*, 2026.

[7] Michal Geyer, Omer Bar-Tal, Shai Bagon, and Tali Dekel. TokenFlow: Consistent Diffusion Features for Consistent Video Editing. In *International Conference on Learning Representations (ICLR)*, 2024.

[8] Haoyang He, Jie Wang, Jiangning Zhang, Zhucun Xue, Xingyuan Bu, Qiangpeng Yang, Shilei Wen, and Lei Xie. OpenVE-3M: A Large-Scale High-Quality Dataset for Instruction-Guided Video Editing. *arXiv preprint arXiv:2512.07826*, 2025.

[9] Hao He, Yinghao Xu, Yuwei Guo, Gordon Wetzstein, Bo Dai, Hongsheng Li, and Ceyuan Yang. CameraCtrl: Enabling Camera Control for Video Diffusion Models. In *International Conference on Learning Representations (ICLR)*, 2025.

[10] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems*, pages 6840–6851, 2020.

[11] Ian Huang, Guandao Yang, and Leonidas Guibas. BlenderAlchemy: Editing 3D Graphics with Vision-Language Models. In *European Conference on Computer Vision (ECCV)*, 2024.

[12] Zeyinzi Jiang, Zhen Han, Chaojie Mao, Jingfeng Zhang, Yulin Pan, and Yu Liu. VACE: All-in-One Video Creation and Editing. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 17191–17202, 2025.

[13] Ozgur Kara, Bariscan Kurtkaya, Hidir Yesiltepe, James M. Rehg, and Pinar Yanardag. RAVE: Randomized Noise Shuffling for Fast and Consistent Video Editing with Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6507–6516, 2024.

[14] Max Ku, Cong Wei, Weiming Ren, Harry Yang, and Wenhu Chen. AnyV2V: A Tuning-Free Framework For Any Video-to-Video Editing Tasks. *Transactions on Machine Learning Research*, 2024.

[15] Zhiyang Liang, Jiawei Zhang, Guocheng Qian, Dongdong Chen, and Ziyu Wan. Code Video Model, 2026. Technical report.

[16] Chenxuan Miao, Yutong Feng, Yi Lu, Yunfeng Yan, Donglian Qi, Shiwei Zhang, Yu Liu, Xi Chen, and Hengshuang Zhao. OmniEdit-Bench: A Comprehensive Benchmark for Instruction-based Video Editing. *arXiv preprint arXiv:2608.05049*, 2026.

[17] MiniMax. MiniMax-H3. Official model and code release, 2026.

[18] Chong Mou, Mingdeng Cao, Xintao Wang, Zhaoyang Zhang, Ying Shan, and Jian Zhang. ReVideo: Remake a Video with Motion and Content Control. In *Advances in Neural Information Processing Systems*, 2024.

[19] OpenSenseNova. SenseNova 6.8 Code2Video. Official model and code release, 2026.

[20] Hao Ouyang, Qiuyu Wang, Yuxi Xiao, Qingyan Bai, Juntao Zhang, Kecheng Zheng, Xiaowei Zhou, Qifeng Chen, and Yujun Shen. CoDeF: Content Deformation Fields for Temporally Consistent Video Processing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8089–8099, 2024.

[21] Chenyang Qi, Xiaodong Cun, Yong Zhang, Chenyang Lei, Xintao Wang, Ying Shan, and Qifeng Chen. FateZero: Fusing Attentions for Zero-shot Text-based Video Editing. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15932–15942, 2023.

[22] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, 2022.

[23] Hanyang Wang, Yimo Cai, Weiliang Chen, Jiawei Chi, Haowen Sun, Qiyu Dai, Yi-Hsin Hung, Xingzhuo Guo, Jinshan Ren, Runmao Yao, Ziwei Liu, Mingsheng Long, Yueqi Duan, Jun Gao, Jiangran Lyu, Fangfu Liu, and Jialong Wu. Code as Worlds: Agentic Discovery of Executable World Representations for Physical Reasoning. *arXiv preprint arXiv:2608.27549*, 2026.

[24] Xiang Wang, Hangjie Yuan, Shiwei Zhang, Dayou Chen, Jiuniu Wang, Yingya Zhang, Yujun Shen, Deli Zhao, and Jingren Zhou. VideoComposer: Compositional Video Synthesis with Motion Controllability. In *Advances in Neural Information Processing Systems*, 2023.

[25] Zhouxia Wang, Ziyang Yuan, Xintao Wang, Tianshui Chen, Menghan Xia, Ping Luo, and Ying Shan. MotionCtrl: A Unified and Flexible Motion Controller for Video Generation. In *ACM SIGGRAPH Conference Papers*, 2024.

[26] Jay Zhangjie Wu, Yixiao Ge, Xintao Wang, Stan Weixian Lei, Yuchao Gu, Yufei Shi, Wynne Hsu, Ying Shan, Xiaohu Qie, and Mike Zheng Shou. Tune-A-Video: One-Shot Tuning of Image Diffusion Models for Text-to-Video Generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7623–7633, 2023.

[27] Yuhui Wu, Liyi Chen, Ruibin Li, Shihao Wang, Chenxi Xie, and Lei Zhang. InsViE-1M: Effective Instruction-based Video Editing with Elaborate Dataset Construction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 16692–16701, 2025.

[28] Yue Yang, Fan-Yun Sun, Luca Weihs, Eli VanderBilt, Alvaro Herrasti, Winson Han, Jiajun Wu, Nick Haber, Ranjay Krishna, Lingjie Liu, Chris Callison-Burch, Mark Yatskar, Aniruddha Kembhavi, and Christopher Clark. Holodeck: Language Guided Generation of 3D Embodied AI Environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16227–16237, 2024.

[29] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding Conditional Control to Text-to-Image Diffusion Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3836–3847, 2023.

[30] Yunzhi Zhang, Zizhang Li, Matt Zhou, Shangzhe Wu, and Jiajun Wu. The Scene Language: Representing Scenes with Programs, Words, and Embeddings. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 24625–24634, 2025.

A. Implementation and Figure Provenance

Program construction. Our implementation adapts the Code Video Model program-and-render workflow to observed source videos. The coding stage uses the Codex CLI with GPT-6 Astra and the `xhigh` reasoning setting. Each case has its own procedural Three.js workspace, source-frame index, rendering helper, and task contract. The agent renders and directly inspects keyframes and encoded-video contact sheets. The renderer evaluates frame i at time $i/24$ through a deterministic `window.reconstruction.seek(t)` interface. A camera-state accessor exports position, quaternion, and field of view after the corresponding time has been evaluated. This metadata is used to validate the procedural output. It is not supplied as a separate camera-control tensor to the video model.

Video synthesis. We use the INT8 H3 implementation from the existing early-step experiment, with a quantized Qwen visual-language encoder. Video-VAE and audio-VAE computation use the established FP16 and FP32 paths, respectively. Sampling uses 20 Euler steps, seed 42, video/audio shifts of 12/3, and the unchanged sigma schedule of that experiment. Both multimodal semantic conditioning and VAE reference conditioning switch from source-plus-proxy to source-only after the fifth step. There is one continuing audiovisual latent trajectory. The source reference latent is shared across the full and reduced conditioning states. All parameters remain frozen.

For throughput, the existing experiment used eight resident single-GPU workers to generate independent cases. This is case-level parallelism; one video does not require the eight workers to participate in a single distributed forward pass. Figure construction only decodes existing videos and does not run H3 again.

Timeline and audio. Source videos are normalized to 24 FPS and proportionally resized and padded to 960×544. If the normalized source has N frames, alignment uses the smallest admissible $N' \geq N$ satisfying $N' = 17m+5$. The additional $N' - N$ frames hold the final source and proxy states. The displayed edited videos are trimmed to the original N-frame interval. Proxies are silent. Generated audio is retained in the original experiment outputs; the present figures and qualitative descriptions concern visual editing.

Showcase selection. The figures use outputs from the `openve_h3_int8_early_step_20261031` experiment. Existing automated judgments were used to shortlist candidates, followed by direct visual inspection of the source, proxy, and both output groups. Selection targets successful illustrations of the requested edits. It is not a random benchmark sample. The main gallery illustrates seagull addition, crescent keyboard removal, person aging, newsroom replacement, clock tower reframing, a celestial portal, Pop Art stylization, and subtitle removal. The introductory figure shows wall clock addition. The same generated output is used wherever a case appears more than once.

Image construction. The supplied figure script decodes exact frame indices from the published source, proxy, and edited MP4s. It preserves the full normalized frame and aspect ratio, then arranges the images with external labels. No image-generation model, retouching, or replacement of visual content is used in making the figures. The accompanying `research/showcase_provenance.json` records video hashes, sampled frame indices, timestamps, prompt hashes, and figure hashes. Figure edit summaries are paraphrases for readability; they are not additional instructions supplied during generation.

B. Conditioning Instructions

Coding-agent task contract. The task provides the source video, timestamped observations, the benchmark edit request, and exact output metadata. Its core requirements are to build a deterministic, fully procedural, editable Three.js scene; implement the requested edit; and retain unedited composition, geometry, occlusion, motion, cuts, and timing. Source imagery is available for observation only. External visual assets and source imagery are not used as rendered textures, plates, overlays, or masks. The contract also requires visual refinement passes covering composition, geometry and occlusion, motion and timing, and semantic legibility. The final encoded video is inspected before it is accepted as a condition.

Reference roles in H3. The no-proxy comparison uses the same source and main editing request, with a reference-role block that identifies only the source video. The proxy-guided group adds the second reference as described below. The original edit request is followed by the source-reference guidance below. This is the reference-role block used in the existing proxy-conditioned showcase runs; it introduces no shot-by-shot target-video description. The numbered video references correspond to their actual input order.

> `source_reference_guidance:` `<Video 1>` is the source video to edit. Preserve its object identities, appearance, color palette, lighting, textures, material treatment, rendering medium, and overall aesthetic, as well as its composition, poses, spatial layout, motion, camera, shot transitions, and timing, except where the main edit prompt requires a change. `<Video 2>` is the structural proxy video showing the requested edit. Use `<Video 2>` as structural guidance for the

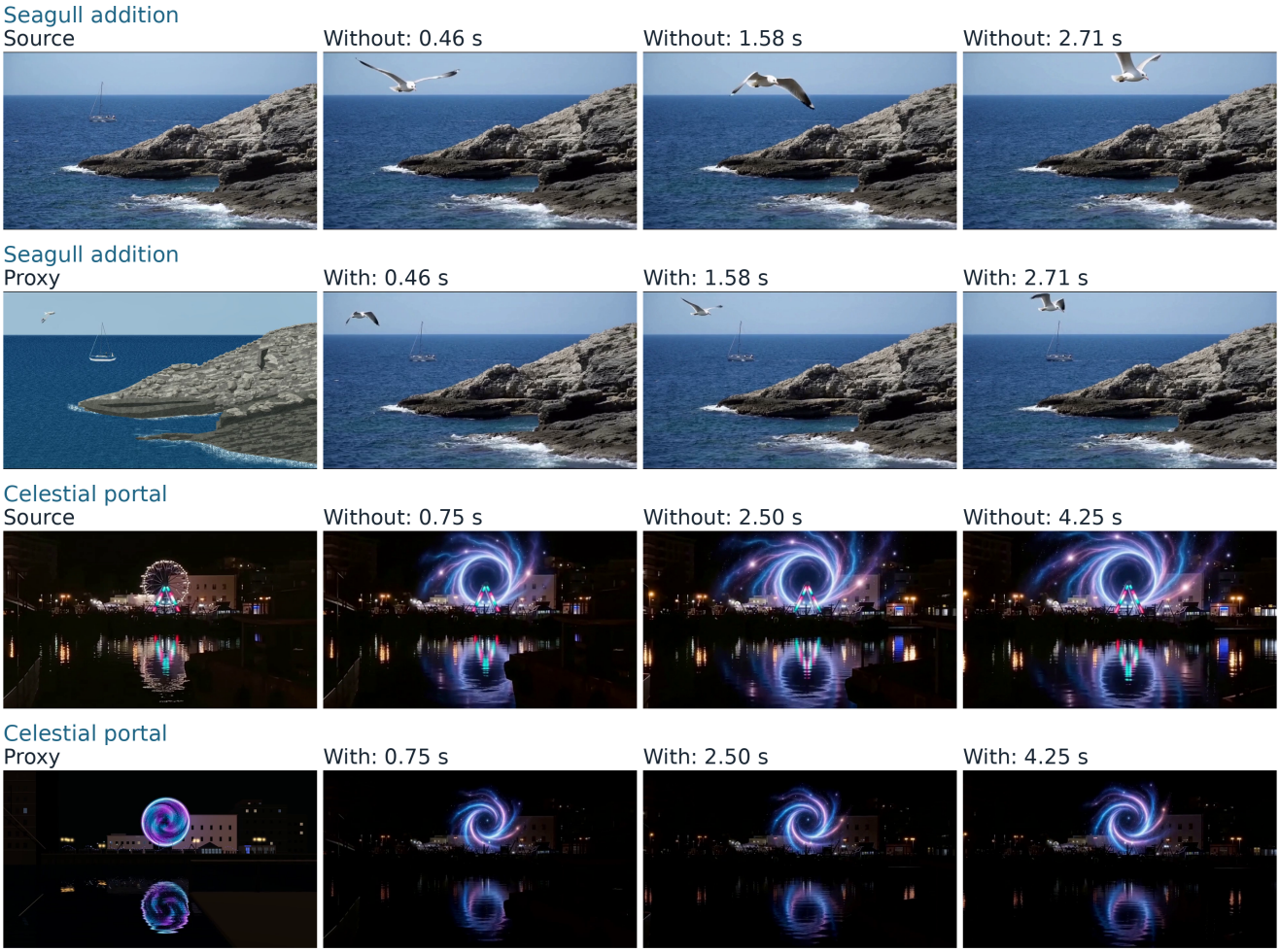


Figure 7. **Additional temporal views of existing showcase videos.** Each case has two rows: a source frame followed by three no-proxy timestamps, and a proxy frame followed by the matching guided timestamps. Top pair: seagull addition. Bottom pair: the celestial portal. All outputs are the same videos used in the gallery.

> requested edit; do not transfer its rendered appearance to the output. Follow the main edit prompt for the requested changes.

The edit request and reference-role text stay in the conditioning prompt throughout the trajectory. After step five, the proxy's visual reference and its associated multimodal conditioning are withdrawn as specified in Eq. (4). For requested stylization or appearance edits, the explicit exception in the preservation instruction allows the intended change. Source preservation is a conditioning objective rather than a pixelwise compositing constraint.